

The CPM Keypoint Model

Mathematics and implementation of `cpm_model_v2.py`, `insect_dataset.py` and `train_heatmap.py`, and how inference inverts them

Project: c:\alignment **Architecture:** Convolutional Pose Machine, 6 stages, $k = 5$ **Date:** 28 July 2026

Every numerical quantity in this document — parameter counts, tensor shapes, receptive fields, loss values, quantisation errors — was measured by executing the code in this repository against the trained checkpoint `epoch-latest.pth`, not taken from the CPM paper. Figures marked *measured* can be reproduced with the snippets shown.

- | | |
|---|---|
| 1. The central design decision | 5. Training |
| 2. Encoding: coordinates $\rightarrow$ heatmaps | 6. Decoding: heatmaps $\rightarrow$ coordinates |
| 3. The architecture | 7. A complete worked example |
| 4. Receptive fields: why six stages | 8. Verified issues |

1. The central design decision

The task is: given a 224×224 image of an insect, output the pixel locations of five anatomical landmarks. The obvious formulation is direct regression — a network ending in a fully-connected layer emitting ten numbers, trained with an L1 or L2 loss on coordinates. Your repository contains exactly that approach in `dataset.py` and `train.py`, and it was abandoned in favour of the heatmap formulation in `insect_dataset.py` and `train_heatmap.py`. Three mathematical reasons justify that choice.

REASON 1 — DIRECT REGRESSION DESTROYS SPATIAL STRUCTURE

A convolutional feature map is *equivariant* to translation: shifting the input shifts the features. Formally, if T_δ is translation by δ and f is a convolutional stack, then

$$f(T_\delta x) = T_{\delta/s} f(x)$$

for stride s . A flatten-then-dense head discards this: the network must relearn, for every output unit, the relationship between feature position and coordinate value. A heatmap head preserves equivariance exactly — moving the landmark moves its predicted blob, with no parameters spent on the correspondence.

REASON 2 — REGRESSION CANNOT EXPRESS UNCERTAINTY OR MULTIMODALITY

Suppose the network is genuinely unsure between two plausible locations p_1 and p_2 . Under a squared-error coordinate loss the optimal single output is the conditional mean:

$$p^* = \arg \min_p \mathbb{E}[\|p - y\|^2] = \mathbb{E}[y] = \frac{1}{2}(p_1 + p_2)$$

— the midpoint, which may be anatomically meaningless. A heatmap can place mass on both hypotheses and let the decoder or a downstream consumer resolve them. It also yields a confidence signal for free (the peak height), which is what the **score** column of the inference CSV reports.

REASON 3 — DENSE SUPERVISION

Coordinate regression provides 10 scalars of training signal per image. The heatmap formulation provides $6 \times 28 \times 28 = 4,704$ supervised values per image. Every spatial position receives a gradient at every step, which is why heatmap methods converge from far less data — a decisive advantage with 599 training images.

THE CONSEQUENCE

Everything that follows is a consequence of this one decision. The dataset must *encode* coordinates into heatmaps (Section 2), the network is an image-to-image map rather than an image-to-vector map (Section 3), the loss is pixel-wise MSE rather than coordinate error (Section 5), and inference must *decode* heatmaps back into coordinates (Section 6). The pipeline is a bijection sandwich: $\text{decode} \circ \text{network} \circ \text{encode}^{-1}$.

2. Encoding: coordinates → heatmaps

Implemented in `insect_dataset.py`, lines 136–158.

2.1 Geometric normalisation

Let the annotated landmark be at (X, Y) in original image pixels and the annotated box be (x_1, y_1, x_2, y_2) . Three operations map it into the network's frame (lines 104–116):

$$\text{translate: } (x_c, y_c) = (X - x_1, Y - y_1) \quad (1)$$

$$\text{scale: } s = \min(224 / w_{\text{crop}}, 224 / h_{\text{crop}}) \quad (2)$$

$$\text{normalise: } (u, v) = (s \cdot x_c / 224, s \cdot y_c / 224) \in [0,1]^2 \quad (3)$$

The min in (2) preserves aspect ratio; the shorter side is then zero-padded on the bottom and right (lines 119–120) to fill the 224×224 canvas. Padding at the bottom-right rather than symmetrically is what keeps the origin at $(0,0)$ and makes the inverse transform in Section 6.2 a pure affine map with no padding offset.

2.2 Discretisation to the heatmap grid

The network predicts on a coarser lattice. With `stride = 224 // 28 = 8`, the grid is 28×28 and the landmark's cell is (line 140–141):

$$c_x = \lfloor u \cdot 28 \rfloor, \quad c_y = \lfloor v \cdot 28 \rfloor$$

The floor is a lossy step and it is the first hard limit on achievable accuracy. It is quantified in Section 7.

2.3 The Gaussian target

Rather than a one-hot indicator — which would give a gradient at exactly one of 784 positions and train very poorly — the target is a Gaussian bump centred on that cell (line 166):

$$H_i(r, c) = \exp(-[(c - c_x)^2 + (r - c_y)^2] / 2\sigma^2), \quad \sigma = 5 \quad (4)$$

Two post-processing steps follow (lines 143–144): values are clipped above at 1, and values below 0.0099 are set to exactly 0. The threshold defines an effective support radius, obtained by inverting (4):

$$r_{\text{max}} = \sigma \sqrt{-2 \ln 0.0099} = 5 \cdot \sqrt{9.234} = \mathbf{15.19 \text{ cells}}$$

This is the critical observation about σ . It is measured in *heatmap cells*, not pixels, so the blob's radius is 15.19 cells on a map that is only 28 cells wide — it covers more than half the map. In the 224-pixel canvas that is a radius of 122 px. Canonical CPM implementations use $\sigma \approx 1\text{--}2$ cells.

Distance from peak	Target value	Note
0 cells	1.0000	peak
1 cell (8 px)	0.9802	almost indistinguishable from the peak
3 cells (24 px)	0.8353	still very high
5 cells (40 px)	0.6065	$= e^{-1/2}$, one standard deviation
10 cells (80 px)	0.1353	
15 cells (120 px)	0.0111	just above the 0.0099 clip

Measured by evaluating $\exp(-d^2/50)$ directly.

The flatness near the peak is the operative problem: a one-cell error changes the target by 2%, so the loss landscape around the correct answer is nearly flat and provides little pressure to localise precisely. This is the mathematical origin of the diffuse predictions seen at inference.

2.4 Channel selection and the background channel

Of the eight annotated landmarks only five are kept (line 147), namely indices `[3, 4, 5, 6, 7]` zero-based, which are landmark IDs 4–8. A sixth channel is then *prepended* (lines 148–150):

$$H_{\text{bg}}(r, c) = 1 - \max_{i \in \{1..5\}} H_i(r, c) \quad (5)$$

This is *not* a softmax normalisation — the six channels do not sum to 1, and each is regressed independently. The background channel acts as an explicit "nothing here" target, giving the network a supervised signal on empty regions instead of leaving them to be inferred from five near-zero channels. It also explains the model's output width: `CPM(k=5)` emits $k + 1 = 6$ channels (`cpm_model_v2.py` line 20).

Measured statistics of one real target tensor:

Channel	max	mean	non-zero cells	Interpretation
0 (background)	1.000	0.6371	779 / 784 (99.4%)	dominant channel
1 (kp4)	1.000	0.1648	432 / 784 (55.1%)	each landmark's blob covers 40–73% of the map
2 (kp5)	1.000	0.1894	575 / 784 (73.3%)	
3 (kp6)	1.000	0.1706	483 / 784 (61.6%)	
4 (kp7)	1.000	0.1364	405 / 784 (51.7%)	
5 (kp8)	1.000	0.1113	310 / 784 (39.5%)	

Measured on `InsectKPDataset("data", "train")[0]`. Overall, 36.6% of target values are exactly 0 and 20.6% exceed 0.5.

2.5 The centre map

A seventh tensor is built at full resolution (lines 152–153) as a Gaussian at the canvas centre:

$$C(r, c) = \exp(-[(c - 112)^2 + (r - 112)^2] / 2 \cdot 5^2)$$

Its integral provides a useful sanity check on the implementation — for a continuous 2-D Gaussian, $\iint \exp(-\|z\|^2/2\sigma^2) dz = 2\pi\sigma^2$:

$$2\pi\sigma^2 = 2\pi \cdot 25 = 157.080 \text{ versus measured sum} = \mathbf{157.080}$$

The centre map tells the network which subject to attend to when several are present — the original CPM's mechanism for multi-person pose estimation. Here it is **identical for every sample**, since the crop always centres one specimen. It therefore carries no per-sample information and acts as a learned constant bias. This is why inference can build it once outside the image loop. It also means the crop step is not optional: the network has been told, by construction, that the subject is at the centre.

3. The architecture

Let $I \in \mathbb{R}^{3 \times 224 \times 224}$ be the image and $C \in \mathbb{R}^{1 \times 224 \times 224}$ the centre map. The model computes six heatmap predictions $H^{(1)} \dots H^{(6)}$, each in $\mathbb{R}^{6 \times 28 \times 28}$, and returns all of them (`cpm_model_v2.py` line 170).

3.1 Downsampling the centre map

$$C' = \text{AvgPool}(C; k=9, s=8, p=1)$$

with output size given by the standard formula

$$\lfloor (224 + 2 \cdot 1 - 9) / 8 \rfloor + 1 = \lfloor 217/8 \rfloor + 1 = 27 + 1 = \mathbf{28}$$

matching the heatmap grid so it can be concatenated at every stage.

3.2 The shared trunk

`_middle` (lines 80–86) computes features reused by every refinement stage:

```
F = pool3( relu( conv3( pool2( relu( conv2( pool1( relu( conv1(I) ) ) ) ) ) ) ) ) )
conv1: 3  -> 128, 9x9      pool: 3x3 stride 2
conv2: 128 -> 128, 9x9     pool: 3x3 stride 2
conv3: 128 -> 128, 9x9     pool: 3x3 stride 2
F in R^(128 x 28 x 28)
```

Three stride-2 pools give total stride $2^3 = 8$, hence $224/8 = 28$. The 9×9 kernels are unusually large by modern standards; Section 4 explains why that matters here.

3.3 Stage 1 — the initial estimate

`_stage1` (lines 68–78) is a separate branch with its *own* copy of the trunk (`conv1_stage1...conv3_stage1` are distinct parameters from `conv*_stage2`), followed by a predictor:

$$H^{(1)} = g_1(I) \quad \text{where } g_1 \text{ ends } 32 \rightarrow 512 (9 \times 9) \rightarrow 512 (1 \times 1) \rightarrow 6 (1 \times 1)$$

Note there is no activation on the final convolution, so $H^{(1)}$ is unbounded. Measured range of the trained stage-6 output on a real sample: $[-0.027, 1.033]$ against a target range of $[0, 1]$. Values slightly outside $[0, 1]$ are expected, not a defect — the model is performing regression, not classification.

3.4 Stages 2–6 — the refinement recurrence

This is the heart of the architecture. Each subsequent stage is a function of the trunk features, the *previous stage's output*, and the centre map:

$$H^{(t)} = g_t(A_t(F) \oplus H^{(t-1)} \oplus C'), \quad t = 2, \dots, 6 \quad (6)$$

where $\oplus$ is channel concatenation and A_t is a 5×5 adapter reducing 128 channels to 32. The concatenated width is

$$32 + (k+1) + 1 = 32 + 6 + 1 = \mathbf{39}$$

which is exactly the input width declared at `cpm_model_v2.py` line 30, `32 + self.k + 2`. Each g_t is then three 11×11 convolutions followed by two 1×1 convolutions.

The essential point about (6) is that **the previous prediction is an input, not just an output**. Stage t can therefore condition landmark 4's position on where it currently believes landmarks 5–8 are. The network learns the joint spatial configuration of the landmark set — an implicit shape model — rather than five independent detectors.

3.5 Parameter budget

Component	Parameters	Share	Comment
Refinement stages 2–6 (Mconv*)	22.93 M	75.2%	the 11×11 kernels dominate
Stage 1 branch	4.38 M	14.4%	includes its private trunk
Shared trunk (conv*_stage2)	2.79 M	9.1%	computed once, reused $5 \times$
Per-stage adapters (conv1_stage3–6)	0.41 M	1.3%	5×5 , $128 \rightarrow 32$
Total	30.51 M	100%	

Measured with `sum(p.numel() for p in CPM(5).parameters())`.

Where the 75% goes is worth seeing explicitly. A single 11×11 convolution with 128 input and 128 output channels costs

$$11 \times 11 \times 128 \times 128 = 1,982,464 \approx \mathbf{1.98 \text{ M}}$$
 parameters

and there are two such layers per stage across five stages, i.e. 19.8 M from those alone. The architecture spends its entire budget buying spatial extent, which Section 4 shows is the point.

3.6 Shape trace

Tensor	Shape	Produced by
image	(1, 3, 224, 224)	dataset
centre map	(1, 1, 224, 224)	dataset
pooled centre map C'	(1, 1, 28, 28)	<code>pool_center</code>
trunk features F	(1, 128, 28, 28)	<code>_middle</code>
stage 1 ... 6 outputs	(1, 6, 28, 28)	<code>_stage1 ... _stage6</code>

Measured by running `CPM(5)` on zero tensors.

4. Receptive fields: why six stages

This is the single most important quantitative argument in the architecture, and it is the reason the design looks so parameter-hungry. The receptive field of a stacked convolution obeys the recurrence

$$R_\ell = R_{\ell-1} + (k_\ell - 1) \cdot j_{\ell-1}, \quad j_\ell = j_{\ell-1} \cdot s_\ell \quad (7)$$

where R is the receptive field in input pixels and j the cumulative stride ("jump"). Applying (7) to this network:

Point in the network	Receptive field	What it can see
Trunk output F	71 px	a local patch — one leg joint, part of the body
Stage 1 output	167 px	most of the specimen, but not the whole canvas
Stage 2 output	343 px	exceeds 224 — the entire image
Stage 3 output	583 px	whole image, with margin
Stage 4 output	823 px	whole image
Stage 5 output	1063 px	whole image
Stage 6 output	1303 px	whole image, $\approx 5.8\times$ over

Measured by applying recurrence (7) to the actual kernel and stride sequence in `cpm_model_v2.py`.

Read the table as the design rationale. Stage 1 is a *local* detector: with a 167 px view it can find things that look like a landmark, but it cannot tell landmark 5 from landmark 7 if they are locally similar, because it cannot see the rest of the animal. From stage 2 onward every prediction is a function of the *entire* image, so the network can enforce global consistency — "this candidate cannot be landmark 8, because landmark 4 is over there."

Each 11×11 convolution contributes $10 \text{ pixels} \times \text{jump } 8 = 80 \text{ px}$ of receptive field, and each stage stacks three of them, adding 240 px per stage. That is precisely why the kernels are large and why the parameter count is concentrated there: **the stages buy context, and context is what disambiguates landmark identity.**

EMPIRICAL CONFIRMATION

Measured per-stage error on a trained sample (Section 5.3) drops sharply from stage 1 to stage 2 — raw MSE 0.0193 $\rightarrow$ 0.0111, a 43% reduction — exactly where the receptive field crosses the image boundary. It then flattens and is essentially converged by stage 4.

5. Training

Implemented in `train_heatmap.py`.

5.1 The loss and intermediate supervision

Lines 159–161 compute:

$$\mathcal{L} = \sum_{t=1}^6 \lambda \cdot \text{MSE}(H^{(t)}, H^{\text{target}}), \quad \lambda = 4704 \quad (8)$$

The sum over *all* six stages — not just the final one — is **intermediate supervision**, and it is what makes a network this deep trainable. Without it, the gradient reaching stage 1 would have to traverse five refinement blocks; with it, every stage receives a direct gradient from the loss:

$$\partial \mathcal{L} / \partial \theta_t = \lambda \partial \text{MSE}(H^{(t)}) / \partial \theta_t + \sum_{t' > t} \lambda \partial \text{MSE}(H^{(t')}) / \partial \theta_t$$

The first term is a short path that cannot vanish regardless of depth. This is the same principle as auxiliary classifiers in GoogLeNet, and it predates residual connections as a solution to the same problem.

5.2 What `heat_weight` actually does

Line 140 sets `heat_weight = 28 * 28 * 6 = 4704`. This number is not arbitrary: it is exactly the number of elements per sample in the heatmap tensor. PyTorch's `MSELoss` defaults to `reduction='mean'`, averaging

over *all* $N \cdot C \cdot H \cdot W$ entries. Multiplying by $C \cdot H \cdot W$ converts that into a sum over the heatmap and a mean over the batch:

$$\lambda \cdot (1 / NCHW) \sum_{n,c,h,w} (\cdot)^2 = (1 / N) \sum_n \sum_{c,h,w} (\cdot)^2$$

So the design intent is the classical CPM objective — total squared error over the heatmap, averaged over the batch. The practical effect is a $\times 4704$ increase in gradient magnitude relative to a bare mean, which must be read together with the learning rate: the default `--lr 1e-4` is small precisely because the loss is large.

5.3 Measured loss values

Running the trained checkpoint on one test sample and evaluating (8) stage by stage:

Stage	Raw MSE	Weighted ($\times 4704$)	Reduction vs. stage 1
1	0.019314	90.854	—
2	0.011073	52.089	−42.7%
3	0.009215	43.348	−52.3%
4	0.008677	40.818	−55.1% (best)
5	0.008992	42.299	−53.4%
6	0.008942	42.064	−53.7%
Σ (reported loss)		311.472	

Measured on `InsectKPDataset("data", "test")[0]` with `epoch-latest.pth`.

For scale, trivial baselines on 20 test samples give a weighted per-stage loss of **749.43** for predicting all zeros and **473.97** for predicting the global mean, against ≈ 42 for the trained stage 6. The model is doing real work.

Two things are visible in this table. The large gain at stage 2 corroborates the receptive-field argument of Section 4. And **refinement saturates at stage 4** — stages 5 and 6 are marginally worse than stage 4 on this sample. Six stages may be more than this problem needs; a 4-stage variant would cut roughly 9 M parameters. That is a single-sample observation, so treat it as a hypothesis worth testing on the full validation set, not as an established result.

5.4 Optimiser and schedule

AdamW (line 33) with a cosine schedule and linear warm-up (`utils.py` lines 200–211):

$$\eta(i) = \eta_{\text{base}} \cdot i / i_{\text{warm}} \quad \text{for } i < i_{\text{warm}} \quad (9)$$

$$\eta(i) = \eta_{\text{min}} + \frac{1}{2}(\eta_{\text{base}} - \eta_{\text{min}})(1 + \cos(\pi \tau)), \quad \tau = (i - i_{\text{warm}}) / (i_{\text{total}} - i_{\text{warm}}) \quad (10)$$

Note the schedule is indexed by *iteration*, not epoch, and is looked up with a global counter `global_it = global_start + it - 1` (line 144). With the defaults `lr=1e-4`, `min_lr=1e-6`, `warmup_epochs=5`, `max_epoch=100`:

Epoch	Learning rate	Phase
0	0.00e+00	warm-up starts from zero
2	4.00e-05	linear ramp
5	1.00e-04	peak; cosine decay begins
50	5.46e-05	half-way, $\approx$ half the peak
100	1.00e-06	floor

5.5 Augmentation as a group action

Lines 126–132 apply two independent flips with probability $\frac{1}{2}$ each. Because a coordinate is normalised to $[0,1]$ over the *whole canvas*, the label transform is the corresponding reflection:

$$\sigma_x: (u, v) \mapsto (1 - u, v) \quad \sigma_y: (u, v) \mapsto (u, 1 - v)$$

This is geometrically correct despite the asymmetric bottom-right padding, because the flip is applied to the full canvas and the normalisation is canvas-relative, so both content and padding reflect together.

The two independent flips generate the **Klein four-group** $V_4 = \{e, \sigma_x, \sigma_y, \sigma_x\sigma_y\}$, each element sampled with probability $\frac{1}{4}$. Note that $\sigma_x\sigma_y$ = rotation by 180° , so the group contains two reflections (which reverse chirality) and one proper rotation (which does not). For specimens on a slide in arbitrary orientation this is a defensible choice; see Section 8.5 for the caveat about landmark identity.

6. Decoding: heatmaps → coordinates

Inference inverts Section 2. Implemented in `infer_unlabeled.py`, lines 53–60 and 219–221.

6.1 Soft-arg-max

The naive decoder is arg max, but it quantises to whole cells — 8 px in canvas space. Instead a weighted average is taken over the k strongest cells:

$$w_j = \exp(s_j) / \sum_{m=1}^k \exp(s_m) \quad (11)$$

$$x_{224} = 8 \cdot \sum_j w_j c_j, \quad y_{224} = 8 \cdot \sum_j w_j r_j \quad (12)$$

where s_j are the top-k heatmap values and (c_j, r_j) their grid coordinates, recovered from the flat index n_j by

$$c_j = n_j \bmod 28, \quad r_j = \lfloor n_j / 28 \rfloor$$

since the flattening is row-major, $n = r \cdot 28 + c$.

THE SOFTMAX IN (11) IS NEARLY UNIFORM

This is a real weakness of the decoder as written. The s_j are raw regression outputs in roughly $[0, 1]$, so the largest achievable weight ratio is

$$\exp(1) / \exp(0) = e \approx 2.718$$

Equation (12) is therefore much closer to an *unweighted mean* of the top-k cells than to a peak-weighted estimate. On well-behaved images this is harmless because the top cells are adjacent — measured difference between `--topk 5` and `--topk 1` on the test images was about 10 px. It fails when a channel is bimodal: the estimate lands between the two peaks, on neither. Introducing a temperature, $w_j \propto \exp(s_j/T)$ with $T \ll 1$, would sharpen it toward a true arg max.

6.2 Inverting the geometric transform

Composing the inverses of (3), (2) and (1) in reverse order gives a pure affine map:

$$X = x_{224} / s + x_1, \quad Y = y_{224} / s + y_1 \quad (13)$$

No padding term appears, which is a direct consequence of the encoder padding only the bottom and right. Had it padded symmetrically, (13) would need an extra offset of $(224 - s \cdot w_{\text{crop}})/2$.

6.3 Effective resolution

One heatmap cell corresponds to $8/s$ original pixels, and s depends on the box. This makes the achievable precision *image-dependent*:

Image	Box side	s	1 cell in original px
GRSM40_2_2025-01-01_253.jpg	376	0.596	13.4
GRSM40_2_2025-01-01_254.jpg	357	0.627	12.8
GRSM40_3__80.jpg	649	0.345	23.2
GRSM40_3__81.jpg	620	0.361	22.1

Measured from `output/unlabeled/keypoints.csv`.

6.4 Dropping the background channel

Before decoding, inference discards channel 0 (`final = final[:, 1:]`). This is mandatory: channel 0 is the background map of equation (5), whose arg max is wherever landmarks are *absent*. Decoding it produces a spurious sixth "landmark", and — worse — shifts every subsequent index by one. This is the defect present in the older `inference.py` at line 195.

7. A complete worked example

Taking a real annotation and pushing it through encode → decode, with no network involved, isolates the precision cost of the representation itself.

```
file          Entomobryomorpha_000049.jpg (2592 x 1944)
box           [527.6, 573.4, 1988.2, 1623.5]    -> crop 1460.5 x 1050.0
landmark 4 raw (946.40, 872.41)
```

Step	Operation	Result	Equation
1	subtract box origin	(418.78, 298.99)	(1)
2	$s = \min(224/1460.5, 224/1050.0)$	0.15337	(2)
3	multiply by s	(64.23, 45.86)	(2)
4	divide by 224	(0.2867, 0.2047)	(3)
5	$\lfloor \cdot \times 28 \rfloor \rightarrow$ grid cell	(8, 5)	§2.2
6	decode, $\times 8$ back to canvas	(64, 40)	(12)
7	$\div s$, add box origin	(944.9, 834.2)	(13)

Comparing step 7 against the original annotation (946.40, 872.41):

round-trip error = (**-1.5**, **-38.2**) original pixels

The x error is small by luck; the y error is large because $0.2047 \times 28 = 5.73$ and the floor at step 5 discards 0.73 of a cell. With a crop this large, $s = 0.153$ means one cell spans 52.2 original pixels, so 0.73 cells is 38 px.

WHAT THIS EXAMPLE ESTABLISHES

This is **not** model error — no network was run. It is the error floor imposed by the 28×28 grid, and it applies to the *training targets* as much as to predictions. The labels the model learns from are themselves quantised this coarsely. Two consequences follow. First, no amount of training can push accuracy below this floor with this stride. Second, the soft-arg-max of Section 6.1 partially recovers sub-cell precision at inference, which is exactly why it exists — but the encoder has no corresponding mechanism, so the targets stay quantised. Encoding the Gaussian at the *unrounded* position (dropping the floor at line 140–141) would cost nothing and remove this asymmetry.

8. Verified issues

Each item below was confirmed by executing the code, not by inspection alone.

8.1 `train_heatmap.py` depends on DDP to run at all

Lines 153–155 move `images` and `targets` to the GPU but pass `centermaps` straight from the loader, still on the CPU. Since `forward` concatenates it with GPU tensors (line 91), this should fail — and it does:

```
>>> m = CPM(5).cuda(); m(img_cuda, centermap_cpu)
RuntimeError: Expected all tensors to be on the same device, but found at least
two devices, cuda:0 and cpu! (when checking argument for argument tensors in
method wrapper_cat)
```

It works in practice only because line 44 wraps the model in `DistributedDataParallel(..., device_ids=[args.gpu])`, and single-device DDP *scatters inputs onto the target device* before calling `forward`. Verified:

```
>>> ddp = DistributedDataParallel(CPM(5).cuda(), device_ids=[0])
>>> ddp(img_cuda, centermap_cpu)[-1].device
device(type='cuda', index=0)      # DDP moved it silently
```

The practical consequence: **you cannot debug this script single-GPU without DDP**. Removing the wrapper — the natural first step when diagnosing a training problem — makes it crash with an error that points at the model rather than at the missing `.cuda()`. Adding `centermaps.cuda(non_blocking=True)` at lines 155 and 211 removes the hidden dependency at no cost.

8.2 `lmk_mask` is computed and discarded

Lines 121–124 build a per-landmark mask that zeroes landmarks 3 and 7 for profile views, but `__getitem__` returns only `image`, `heatmaps`, `centermap` at line 160. The mask never reaches `train_heatmap.py`, whose loss at line 161 is unmasked. The older coordinate pipeline did use it (`train.py` line 155), so this appears to have been lost in the port.

The concrete effect runs through lines 93–95, which fill missing profile landmarks by duplication:

```
lmks[3-1] = lmks[2-1]      # kp3 := kp2 - later discarded by the channel selection, harmless
lmks[7-1] = lmks[5-1]      # kp7 := kp5 - RETAINED as channel 4, and trained on
```

328 of the 755 annotated images are profile views, so for **43% of the training set the model is taught that landmark 7 coincides with landmark 5**. Predictions are consistent with this: on the inference test images `kp5` and `kp7` land about 9 px apart on one and share an identical `y` on another. Treat `kp7` as unreliable and do not use the `kp5–kp7` distance as a morphometric feature.

8.3 $\sigma = 5$ is very broad for a 28×28 map

As derived in Section 2.3, the support radius is 15.19 cells on a 28-cell map, and the target changes by only 2% for a one-cell displacement. The gradient signal for precise localisation is correspondingly weak. Reducing σ to 1–2 cells is the standard setting and would sharpen predictions, but requires retraining and may need a lower learning rate, since sparser targets increase gradient variance.

8.4 Argument order at the gaussian call sites

The function signature is `gaussian(x, y, H, W)` with H rows and W columns, and the body is self-consistent — verified by constructing a deliberately non-square map, where `gaussian(x=5, y=20, H=40, W=20)` correctly peaks at (row 20, col 5). The call sites, however, pass width into H and height into W : line 142 passes `(w//stride, h//stride)` and line 153 passes `(w, h)`. This is harmless today *only* because the canvas is square, so both arguments are equal. It would silently transpose every target if `target_size` ever became non-square.

8.5 Flips do not relabel landmarks — verify this against your protocol

The flip augmentation of Section 5.5 transforms coordinates but leaves landmark *identities* untouched. In human pose and face alignment this is a known error: a horizontal mirror maps the left elbow onto the right elbow, so the label indices must be swapped as well, otherwise the network receives contradictory targets for the same visual pattern.

Whether this applies here depends on anatomy the repository does not record — the annotations store only integer IDs. There is circumstantial reason for concern: landmarks 3 and 7 are precisely the two absent from profile views (deducible from `kp_6map8`, whose image is `{1,2,4,5,6,8}`), which is what one expects of a bilaterally symmetric pair. **If any of landmarks 4–8 form a left/right pair, the flip augmentation is injecting label noise.** The check is quick: plot several annotated specimens with landmark IDs drawn, and see whether mirroring maps the set onto itself with identities preserved. If not, add the corresponding index permutation inside the flip branches.

8.6 The train/test split is deterministic by file order

Lines 50–63 sort images by class, then take the first 80% for training and the last 20% for testing, with no shuffling. Filenames are sequential (`Entomobryomorpha_000048, ...049, ...060`), which for a microscopy session usually means consecutive frames of the same or adjacent specimens. Near-duplicate frames straddling the 80% boundary would leak between the splits and make validation loss optimistic. Worth verifying before quoting any validation number; grouping the split by specimen rather than by index would settle it.

8.7 Refinement saturates before stage 6

From the measured table in Section 5.3, stage 4 achieves the lowest error and stages 5–6 are marginally worse. If this holds across the validation set, a 4-stage model would save roughly 9 M parameters and two-thirds of the refinement compute at no accuracy cost. Single-sample evidence — treat as a hypothesis to test, not a conclusion.

9. Summary of the pipeline

```
ANNOTATION (human)      kp_data.pkl: (x, y, id) per landmark, plus a box

    | insect_dataset.py
    v
ENCODE                  crop -> resize by s=min(224/w, 224/h) -> pad bottom-right
                        (x,y) -> normalise -> floor to 28x28 cell -> Gaussian, sigma=5
                        select channels [3..7] -> prepend bg = 1 - max
                        TARGET: (6, 28, 28)          CENTRE MAP: (1, 224, 224)

    | cpm_model_v2.py
    v
NETWORK                trunk F (128,28,28), RF 71 px
                        H1 = g1(I)                      RF 167 px (local)
                        Ht = gt(A(F) + H_{t-1} + C'), t=2..6 RF 343 -> 1303 px (global)
                        returns all six: 6 x (6, 28, 28)

    | train_heatmap.py
    v
TRAIN                  L = SUM_t MSE(H_t, target) * 4704    <- intermediate supervision
                        AdamW, cosine LR 1e-4 -> 1e-6, 5-epoch warm-up
                        augment: Klein four-group of flips

    | infer_unlabeled.py
    v
DECODE                 take H6, DROP channel 0 (background)
                        soft-arg-max over top-k -> (x224, y224)
                        X = x224/s + x1, Y = y224/s + y1
                        OUTPUT: original-image pixels + peak score
```

REPRODUCING THE MEASUREMENTS

Parameter counts and shapes: instantiate `CPM(5)` and inspect `named_parameters()`. Receptive fields: apply recurrence (7) to the kernel and stride sequence. Loss values: load `epoch-latest.pth`, run one `InsectKPDataset` sample, and evaluate `MSELoss()(pred, target) * 4704` per stage. The round-trip example reads `kp_data.pkl` index 1 directly. All were executed with the `insect_model` conda environment (torch 1.12.1+cu113).